Oleh Prostakov
Anastasiia Mazur

Time Managemem t

Recommender Systems

Dataset

MovieLens 1M

- 1M interactions
- 6k movies
- 4k users
- 95% sparsity
- 20% of users produce 65% of interactions

Temporal Splits

Per-User

- Used for ‘profile-building’ models
- Every user end up with train and test splits

Global

- Used for ‘aggregating’ models
- Provides more realistic environment
- A lot of users end up with no test set

Extensions

How to make movie data more informative?

- Extract descriptions from TMDB (free version of IMDB
- Vectorize them
  - We used SentenceBERT, but something simpler, like TF-IDF should work as well
- Combine component-embeddings into movie-embeddings
  - We used concat+PCA

Metrics

What we scored for:

- NDCG (**primary**)
- Precision
- Recall
- MAP (mean average precision)
- MRR (mean reciprocal rank)

Relevance threshold: 4

---

*all @10

Models

We evaluated a bit of everything for this project:

- Similarity-based recommender*
- ALS
- FunkSVD
- Heuristic
- BPR
- Hybrid
- WideDeep
- Two Towers*

---

*on the per-user split

Similarity-Based (per-user split)

The idea is simple. Build history-based profiles. We went a bit further:

- Pearson instead of cosine similarity
- Hybrid scoring (biased for the global rating of the movie)
- Recency decay
- Neighbor enrichment
- Gradient Boosted Regression re-ranker
- FAISS as vector storage

Final NDCG:
0.128

Our target to beat:
0.04

ALS (global split)

Factorizes the user-item rating matrix into low-rank user and item factor matrices. Learns through ALS

- Training:
  - Number of factors - 50:200
  - 15 iterations
- Findings:
  - More factors (higher rank) consistently improve ranking
  - Lower RMSE does not translate to better NDCG

Final NDCG:

0.035

FunkSVD (global split)

Learns user and item latent factors via SGD on observed ratings

- Training:
  - Number of factors - 50:150
  - 15 iterations
- Findings:
  - 50-factor model outperformed larger ones (better generalization)
  - Can handle high LR (0.01)

Final NDCG:

0.091: >> ALS

Heuristics (global split)

Baselines ranking items by popularity, rating averages, or graph propagation over the item co-interaction graph.

- Models:
  - Popularity: count, recency
  - Rating: mean rating, Bayesian shrinkage
  - Graph: item graph, PageRank, PPR
- Findings:
  - Graph models generalize best
  - Mean-rating and Bayesian heuristics underperform
  - **80% of users are skipped during evaluation**

Best NDCG:

0.296: >> FunkSVD

BPR (global split)

For each user, learn to rank items they liked higher than sampled negatives

- Training:
  - Number of factors - 64
  - 20 epochs
- Findings:
  - Uniform negatives outperform popularity-weighted negatives
  - Sampling strategy matters more than pool choice (unseen vs non_positive)
  - Tail-item performance is near zero

NDCG:

0.296 (same as graph models)

Hybrid

Retrieve from CF + CB, then rerank with a learning-to-rank model to combine signals

- Stage 1 (retrieve):
  - BPR (CF): 64 factors, 20 epochs → top-400
  - Content-based (SBERT + FAISS) → top-120 → Union 520
- Stage 2 (rerank):
  - LambdaMART on 16 features + (opt) CF blend
  - OR RRF (no training)
- Findings:
  - CB adds candidates, CF keeps precision, CF×CB is most informative

NDCG:

0.143/0.296 (same as graph models and bpr)

WideDeep

Wide linear component (user/item/demographic biases) for memorization plus deep MLP over user, item, and side-feature embeddings for generalization

- Findings:
  - Best config differs by split: smaller model (e64, h128x64) wins on global, larger (e96, h192x96) wins on per-user
  - Side features (demographics, genres) are directly included in scoring, unlike MF/BPR

NDCG:
0.128/0.291

Two Towers (per-user)

Like similarity-based, with a couple of adjustments:

- User-tower:
  - Self-attention between history items
  - Cross-attention with user profiles
  - Projection into dim=128 (instead of PCA)
- Item-tower:
  - Projection into dim=128
- Loss: InfoNCE, batch_size=512
- **No reranker**
- **Cosine similarity**

Final NDCG:

0.137

Online Evaluation

Multi-Armed bandits

- Experiment:
  - Model A has slightly higher NDCG than Model B. Does Model A produce significantly-better recommendations?
- Metric:
  - Click-through rate, with NDCG as proxy
- Bandits:
  - A/B test (we can reframe it as a bandit)
  - Epsilon-greedy
  - Thompson Sampling

Leaderboards

Per-user temporal split

Model	NDCG@10	Precision@10	Recall@10	MRR@10
Content-based (Pearson + recency)	0.053	0.039	0.020	—
Content-based GBR re-ranker (full pipeline)	0.128	0.110	0.052	—
Hybrid LambdaMART (blend=0.6)	0.126	0.108	0.050	0.231
Wide & Deep (wd_e96_h192x96_neg3)	0.128	0.109	0.049	0.239
Two-tower transformer (epoch 15)	0.137	0.100	0.063	0.217
Hybrid RRF no-ranker (α=0.85)	**0.143**	**0.123**	**0.058**	**0.265**

Global temporal split

Model	NDCG@10	Precision@10	Recall@10	MRR@10	Eval set
Item-Item CF (adj cosine, k=20)	0.030	0.029	0.006	—	val
ALS (200 factors, reg=0.1)	0.035	0.030	0.005	—	val
FunkSVD (50 factors, lr=0.01)	0.091	0.083	0.021	—	val
popularity_count	0.217	0.193	0.042	0.351	test
Wide & Deep (wd_e64_h128x64_neg2)	0.221	0.199	0.043	0.347	test
item_graph (α=0.85, steps=1)	0.235	0.213	0.049	0.368	test
BPR (uniform, unseen)	0.239	0.212	0.050	0.385	test
Hybrid LambdaMART (blend=0.6)	**0.248**	**0.217**	**0.055**	**0.393**	**test**

Q&A